Assignment 4

Graph Optimization, Greedy Methods, and Dynamic Programming

Algorithms Design and Analysis

Kyal Sin Lin Lett — st126112

April 18, 2026

Theme: Resilient logistics and infrastructure planning for a multi-campus university network.

Contents

1 Part A — Theory and Algorithmic Reasoning **2**
1.1 A1 — Graph Representation . . . 2
1.2 A2 — BFS vs. Weighted Shortest Path . . . 2
1.3 A3 — Dijkstra Failure Case with Negative Edges . . . 3
1.4 A4 — Greedy vs. Dynamic Programming . . . 3

2 Part B — Graph Algorithms **4**
2.1 B1 — BFS: Unweighted Reachability and Hop Distance . . . 4
2.2 Output: . . . 4
2.3 B2 — DFS: Structure Analysis . . . 5
2.4 Output: . . . 5
2.5 B3 — Dijkstra and Bellman-Ford . . . 6
2.6 Output: . . . 6
2.7 B4 — Floyd-Warshall All-Pairs Shortest Paths . . . 7
2.8 Output: . . . 8

3 Part C — Greedy Network Design **8**
3.1 C1 — MST via Prim's Algorithm . . . 8
3.2 Output: . . . 9
3.3 C2 — Greedy Correctness Discussion . . . 9

4 Part D — Dynamic Programming Scheduling **10**
4.1 D1 — Greedy Interval Scheduling (Maximise Job Count) . . . 10
4.2 Output: . . . 10
4.3 D2 — Weighted Interval Scheduling (DP) . . . 11
4.4 Output: . . . 12
4.5 D3 — Greedy Failure Example for Weighted Scheduling . . . 12

Part A — Theory and Algorithmic Reasoning

A1 — Graph Representation

Case 1: Sparse graph with $V = 100{,}000$ vertices and $E = 300{,}000$ edges. An **adjacency list** is the appropriate choice.

- **Memory.** An adjacency list consumes $O(V + E) = O(400{,}000)$ space. An adjacency matrix would require $O(V^2) = O(10^{10})$ entries – roughly 75 GB for 64-bit integers. The list representation is orders of magnitude more compact.
- **Edge lookup.** $O(\deg(u))$ per query in a list vs. $O(1)$ in a matrix. For sparse graphs $\deg(u) \ll V$, so the list is still fast enough in practice.
- **Neighbour traversal.** $O(\deg(u))$ for both representations. In the sparse case, the list visits only the actual neighbours rather than scanning all V columns.
- **Overall suitability.** All classic sparse-graph algorithms (BFS, DFS, Dijkstra, Bellman-Ford, Prim) are designed around adjacency lists. The matrix is simply infeasible at this scale.

Case 2: Dense graph with $V = 4{,}000$ vertices. An **adjacency matrix** is the appropriate choice.

- **Memory.** $O(V^2) = O(16 \times 10^6)$ entries – about 128 MB for 64-bit values, which is manageable. A dense graph has $E \approx V^2$, so the adjacency list uses roughly the same memory without the $O(1)$-lookup benefit.
- **Edge lookup.** $O(1)$ in the matrix: check `A[u][v]` directly. Useful for algorithms like Floyd-Warshall.
- **Neighbour traversal.** $O(V)$ in both representations when the graph is dense (nearly every column is non-zero anyway).
- **Overall suitability.** When $E \sim V^2$, the matrix's constant-time lookup and cache locality make it preferable.

Table 1: Adjacency List vs. Adjacency Matrix comparison.

Criterion	Adjacency List	Adjacency Matrix
Space	$O(V + E)$	$O(V^2)$
Edge lookup	$O(\deg(u))$	$O(1)$
Neighbour traversal	$O(\deg(u))$	$O(V)$
Best when	Sparse graph ($E \ll V^2$)	Dense graph ($E \sim V^2$)

A2 — BFS vs. Weighted Shortest Path

When BFS returns a correct shortest path. BFS computes shortest paths *in terms of hop count* (number of edges traversed). It is therefore correct whenever all edge weights are equal – most importantly, on *unweighted* graphs where every edge has implicit weight 1.

When BFS does not return a correct shortest path. BFS is incorrect on *weighted* graphs where edges carry different costs. Because BFS expands vertices in order of hop count rather than path weight, it can "commit" to a path with fewer hops that has a higher total cost.

Small example. Consider the graph:

$$S \xrightarrow{1} A, \quad S \xrightarrow{10} B, \quad A \xrightarrow{1} B.$$

BFS from S: both A and B are reachable in 1 hop directly from S. BFS reports $\text{dist}(S, B) = 1$ (direct edge). But the weighted shortest path is $S \to A \to B$ with cost $1 + 1 = 2$, which is less than the direct edge weight of 10. Here BFS gives the wrong *weighted* cost while being correct about hop count.

Conversely, if we ask for the minimum-weight path in this graph, BFS (on the unweighted version) reports $S \to B$ (1 hop) but the weight-optimal path is $S \to A \to B$ (cost 2 vs. 10). BFS cannot detect this because it ignores edge weights.

A3 — Dijkstra Failure Case with Negative Edges

Constructed graph.

$$S \xrightarrow{3} A, \quad S \xrightarrow{4} B, \quad B \xrightarrow{-5} A.$$

Dijkstra's behaviour (incorrect).

1. Initialise: $d[S] = 0$, $d[A] = \infty$, $d[B] = \infty$.
2. Pop S: relax $S \to A$ ($d[A] = 3$) and $S \to B$ ($d[B] = 4$). Heap: $\{(3, A), (4, B)\}$.
3. Pop A (smallest key = 3): A is **finalised** with distance 3. A has no outgoing edges.
4. Pop B: relax $B \to A$: $d[A] = \min(3, 4 + (-5)) = \min(3, -1) = -1$. However, A was already removed from the priority queue and marked as finalised, so this update is never propagated.
5. Dijkstra reports $d[A] = 3$. The correct answer is -1 (via $S \to B \to A$).

Why Dijkstra fails. Dijkstra's greedy invariant assumes that once a vertex is extracted from the min-heap its tentative distance is optimal. Negative edges violate this: a later, seemingly longer path can "shortcut" through a negative edge and arrive cheaper.

Why Bellman-Ford works. Bellman-Ford relaxes *all* edges in every pass, with no finalisation step. After $|V| - 1$ passes it is guaranteed to have propagated every shortest path (assuming no negative cycle). On the same graph:

- Pass 1: relax $S \to A$ ($d[A] = 3$), $S \to B$ ($d[B] = 4$), $B \to A$ ($d[A] = -1$).
- Pass 2: no further improvement.

Bellman-Ford correctly reports $d[A] = -1$.

A4 — Greedy vs. Dynamic Programming

Problem where greedy is optimal: Activity Selection (maximise job count). Sort jobs by finish time and greedily accept a job whenever it does not overlap with the last accepted job. This is provably optimal for *maximising the number* of non-overlapping jobs. The exchange argument shows that replacing any greedily chosen job with another can only worsen or maintain the count (see Part D1 for the implementation).

Superficially similar problem where greedy fails: Weighted Interval Scheduling (maximise total reward). Consider the jobs:

$$A = [1, 2], \text{ reward} = 1 \qquad B = [1, 10], \text{ reward} = 100.$$

Both start at time 1 and are mutually incompatible. Sorted by finish time: A (finishes at 2) before B (finishes at 10).

- **Greedy (earliest finish):** selects A (last finish = 2), then checks B: $\text{start}(B) = 1 < 2$ – skip. Result: $\{A\}$, reward = 1.

- **DP optimal:** select only B, reward = 100.

The greedy strategy picks the job that finishes earliest, but that job has negligible reward compared to a longer, higher-value job. Only DP considers all combinations and finds the globally optimal subset.

Part B — Graph Algorithms

All Part B algorithms operate on `data/roads.csv`: a directed weighted graph with vertices $\{A, B, C, D\}$ and edges:

$$A \to B\ (4), \quad A \to C\ (2), \quad C \to B\ (1), \quad B \to D\ (5), \quad C \to D\ (8).$$

B1 — BFS: Unweighted Reachability and Hop Distance

Approach. Edge weights are ignored; the graph is treated as unweighted. BFS from source A computes the minimum number of hops to every reachable vertex. Parent pointers enable path reconstruction in $O(|path|)$.

```python
def bfs(graph, vertices, source):
    dist   = {v: -1   for v in vertices}
    parent = {v: None for v in vertices}
    dist[source] = 0
    queue = deque([source])

    while queue:
        u = queue.popleft()
        for v in graph.get(u, []):
            if dist[v] == -1:           # not yet visited
                dist[v]   = dist[u] + 1
                parent[v] = u
                queue.append(v)

    return dist, parent


def reconstruct_path(parent, source, target):
    path = []
    cur  = target
    while cur is not None:
        path.append(cur)
        cur = parent[cur]
    path.reverse()
    return path if (path and path[0] == source) else None
```

Listing 1: Core BFS logic (src/b1_bfs.py).

Output:

```
BFS from source: A
Target      Hops        Path
---------------------------------------------
A           0           A
B           1           A -> B
C           1           A -> C
D           2           A -> B -> D
```

Both B and C are one hop from A. D requires two hops (via B). All four vertices are reachable.

B2 — DFS: Structure Analysis

Approach. DFS assigns discovery and finish timestamps to each vertex and classifies every edge according to its relationship in the DFS forest.

```python
def dfs_visit(u):
    color[u] = 'gray';  time[0] += 1;  disc[u] = time[0]

    for v in graph.get(u, []):
        if color[v] == 'white':
            parent[v] = u
            edge_classes.append((u, v, 'TREE'))
            dfs_visit(v)
        elif color[v] == 'gray':
            edge_classes.append((u, v, 'BACK'))
        else:                           # black
            if disc[u] < disc[v]:
                edge_classes.append((u, v, 'FORWARD'))
            else:
                edge_classes.append((u, v, 'CROSS'))

    color[u] = 'black';  time[0] += 1;  fin[u] = time[0]
```

Listing 2: DFS visit routine (src/b2_dfs.py).

Output:

```
DFS Structure Analysis
==============================================
Vertex      disc      fin       parent
----------------------------------------------
A           1         8         None
B           2         5         A
C           6         7         A
D           3         4         B

Edge Classification:
-----------------------------------
  A -> B  :  TREE
  B -> D  :  TREE
  A -> C  :  TREE
  C -> B  :  CROSS
  C -> D  :  CROSS

Discussion:
  No back edges: graph is a DAG (no directed cycles).
  Cross edges [('C','B'),('C','D')]: reach already-finished subtrees,
  confirming the graph has multiple paths without creating cycles.
  All 4 vertices discovered from one DFS root -> weakly connected.
```

Discussion.

- **Cycles.** No back edges are found, so the graph is a directed acyclic graph (DAG). This is consistent with the road network having no circular routes.
- **Connectivity.** DFS from A (the alphabetically first vertex) reaches all four vertices without needing to restart – the graph is weakly connected.
- **Traversal order.** A is the root; B and its subtree (D) are explored first ($A \to B \to D$); C is explored last, contributing two cross edges to already-finished vertices B and D. Cross edges confirm alternative paths between vertices that do not create cycles.

B3 — Dijkstra and Bellman-Ford

```python
def dijkstra(graph, vertices, source):
    dist   = {v: float('inf') for v in vertices}
    parent = {v: None         for v in vertices}
    dist[source] = 0
    pq = [(0, source)]

    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:         # stale heap entry
            continue
        for v, w in graph.get(u, []):
            new_dist = dist[u] + w
            if new_dist < dist[v]:
                dist[v]   = new_dist
                parent[v] = u
                heapq.heappush(pq, (new_dist, v))

    return dist, parent
```

Listing 3: Dijkstra's algorithm (src/b3_shortest_paths.py).

```python
def bellman_ford(edges, vertices, source):
    dist   = {v: float('inf') for v in vertices}
    parent = {v: None         for v in vertices}
    dist[source] = 0

    n = len(vertices)
    for _ in range(n - 1):
        updated = False
        for u, v, w in edges:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v]   = dist[u] + w
                parent[v] = u
                updated   = True
        if not updated:         # converged early
            break

    # One extra pass: any improvement means a negative cycle exists.
    has_neg_cycle = any(
        dist[u] != float('inf') and dist[u] + w < dist[v]
        for u, v, w in edges
    )
    return dist, parent, has_neg_cycle
```

Listing 4: Bellman-Ford algorithm (src/b3_shortest_paths.py).

Output:

```
Dijkstra     shortest paths from 'A':
Target     Distance     Path
--------------------------------------------------
A          0            A
B          3            A -> C -> B
C          2            A -> C
D          8            A -> C -> B -> D

Bellman-Ford     shortest paths from 'A':
Target     Distance     Path
--------------------------------------------------
A          0            A
```

```
B           3             A -> C -> B
C           2             A -> C
D           8             A -> C -> B -> D

Negative cycle detected: False
```

Discussion. Both algorithms agree because `roads.csv` has no negative weights.

- **Dijkstra** runs in $O((V + E) \log V)$ with a binary heap. Its greedy invariant (always relax the nearest unfinished vertex) is correct only when all weights are non-negative. On `roads.csv` the optimal route to B uses the cheaper detour $A \to C \to B$ (cost 3) instead of the direct edge $A \to B$ (cost 4).
- **Bellman-Ford** runs in $O(VE)$. It performs $|V| - 1$ full relaxation passes and then one extra pass to check for negative cycles. It is slower but handles negative-weight edges and detects negative cycles.
- **When to prefer each.** Use Dijkstra for large sparse graphs without negative weights (e.g., road-navigation with positive distances). Use Bellman-Ford when negative weights appear (e.g., financial graphs with gains and losses) or when negative-cycle detection is required.

B4 — Floyd-Warshall All-Pairs Shortest Paths

```python
def floyd_warshall(vertices, edges):
    n   = len(vertices)
    idx = {v: i for i, v in enumerate(vertices)}

    dist = [[INF]  * n for _ in range(n)]
    nxt  = [[None] * n for _ in range(n)]

    for i in range(n):
        dist[i][i] = 0

    for u, v, w in edges:
        i, j = idx[u], idx[v]
        if w < dist[i][j]:
            dist[i][j] = w
            nxt[i][j]  = j      # next-hop index

    for k in range(n):
        for i in range(n):
            for j in range(n):
                via = dist[i][k] + dist[k][j]
                if via < dist[i][j]:
                    dist[i][j] = via
                    nxt[i][j]  = nxt[i][k]    # inherit next hop

    has_neg_cycle = any(dist[i][i] < 0 for i in range(n))
    return dist, nxt, has_neg_cycle, idx


def reconstruct_path(vertices, nxt, idx, src, dst):
    i, j = idx[src], idx[dst]
    if nxt[i][j] is None:
        return None
    path = [src]
    while i != j:
        i = nxt[i][j]
        path.append(vertices[i])
    return path
```

Listing 5: Floyd-Warshall with path reconstruction (src/b4_floyd_warshall.py).

Output:

```
Floyd-Warshall All-Pairs Shortest Paths
==================================================
Negative cycle detected: False

Distance matrix (INF = no path):
      A       B       C       D
A     0       3       2       8
B   INF       0     INF       5
C   INF       1       0       6
D   INF     INF     INF       0

Path reconstruction (all reachable pairs):
--------------------------------------------------
A -> B: dist=3, path=A -> C -> B
A -> C: dist=2, path=A -> C
A -> D: dist=8, path=A -> C -> B -> D
B -> A: unreachable
B -> C: unreachable
B -> D: dist=5, path=B -> D
C -> A: unreachable
C -> B: dist=1, path=C -> B
C -> D: dist=6, path=C -> B -> D
D -> A/B/C: unreachable
```

Path reconstruction. The `nxt[i][j]` table stores the index of the next vertex after i on the shortest path to j. To reconstruct path $A \to D$: $\texttt{nxt}[A][D] = C$, then $\texttt{nxt}[C][D] = B$, then $\texttt{nxt}[B][D] = D$ – yielding $A \to C \to B \to D$ with cost 8.

Negative-cycle detection. If `dist[i][i] < 0` after the algorithm completes, vertex i lies on a negative-weight cycle. No such cycle exists here.

Reflection: Floyd-Warshall vs. repeated Dijkstra.

- Floyd-Warshall is $O(V^3)$, simple to implement, and directly produces the full $V \times V$ distance matrix. It is a good choice when V is small (here $V = 4$) or the graph is dense.
- Repeated Dijkstra (once per source) is $O(V \cdot (V + E) \log V)$. For large sparse non-negative-weight graphs, this is typically faster.
- Floyd-Warshall is *not* a good choice when V is large (e.g., $V = 100{,}000$ would require 10^{15} operations) or when only a few source-to-destination pairs are needed.

Part C — Greedy Network Design

Part C uses `data/infra.csv`: an undirected weighted graph with vertices $\{A, B, C, D\}$ and edges A-B (7), A-C (2), B-C (5), B-D (6), C-D (4).

C1 — MST via Prim's Algorithm

```python
def prim(graph, vertices, start):
    in_mst = set()
    key    = {v: float('inf') for v in vertices}
    parent = {v: None         for v in vertices}
```

```python
    key[start] = 0
    pq = [(0, start)]

    mst_edges  = []
    total_cost = 0

    while pq:
        cost, u = heapq.heappop(pq)
        if u in in_mst:        # stale heap entry
            continue
        in_mst.add(u)

        if parent[u] is not None:
            mst_edges.append((parent[u], u, cost))
            total_cost += cost

        for v, w in graph.get(u, []):
            if v not in in_mst and w < key[v]:
                key[v]    = w
                parent[v] = u
                heapq.heappush(pq, (w, v))

    return mst_edges, total_cost
```

Listing 6: Prim's MST (src/c1_prim.py).

Output:

```
Prim's Minimum Spanning Tree
====================================
Starting vertex: A

Edge            Weight
-------------------------
A -- C          2
C -- D          4
C -- B          5
-------------------------
Total MST cost: 11

MST spans all 4 vertices (3 edges) -- valid spanning tree.
All MST edge weights are distinct -> the MST is unique.
```

MST uniqueness. When all edge weights are distinct, the MST is unique (provable by contradiction using the cut property). All five edges in `infra.csv` have distinct weights (2, 4, 5, 6, 7), so the MST $\{A\text{-}C, C\text{-}D, C\text{-}B\}$ with cost 11 is the only MST.

Had two edges tied for the same weight, multiple valid MSTs could exist; in that case we would report both and note they share the same total cost.

C2 — Greedy Correctness Discussion

Why MST is a greedy problem. MST algorithms are greedy in the sense that each step makes a locally optimal choice – add the cheapest edge that does not violate the spanning-tree constraints – and this local choice is provably globally optimal. The correctness rests on two structural properties:

- **Cut property.** For any cut $(S, V \setminus S)$ of the graph, the minimum-weight crossing edge belongs to some MST. Prim's algorithm always adds the cheapest edge that crosses the

frontier between the in-MST and not-yet-in-MST vertices, respecting this property at every step.

- **Cycle property.** The maximum-weight edge in any cycle cannot belong to any MST (it can always be removed without disconnecting the tree).

Why naïve "pick the globally cheapest remaining edge" fails without cycle checking. Consider the graph A-B (2), B-C (3), A-C (4). A valid MST contains edges $\{A\text{-}B, B\text{-}C\}$ (cost 5).

If we pick the cheapest remaining edge at each step *without checking for cycles*:

1. Pick A-B (2). Selected: $\{A\text{-}B\}$.
2. Pick B-C (3). Selected: $\{A\text{-}B, B\text{-}C\}$.
3. Pick A-C (4). **Without cycle checking**, we add this edge too.

Result: $\{A\text{-}B, B\text{-}C, A\text{-}C\}$, which is a *cycle* – no longer a tree. Cycle checking (as done by Kruskal's algorithm using a Union-Find structure) is essential to ensure the result remains a tree.

Counterexample (concrete). Use the same three-vertex graph above. The naïve greedy without cycle detection selects all three edges (A-B, B-C, A-C) at costs 2, 3, 4 = total 9. A valid spanning tree needs only two edges; the MST has cost 5. The naïve result is both more expensive and structurally invalid (it contains a cycle).

Part D — Dynamic Programming Scheduling

Part D uses `data/jobs.csv`:

Job	Start	Finish	Reward
J1	1	4	5
J2	3	5	2
J3	0	6	8
J4	4	7	6
J5	3	8	7

D1 — Greedy Interval Scheduling (Maximise Job Count)

```python
def greedy_interval_scheduling(jobs):
    sorted_jobs  = sorted(jobs, key=lambda j: j['finish'])
    selected     = []
    last_finish  = -1     # sentinel

    for job in sorted_jobs:
        if job['start'] >= last_finish:
            selected.append(job)
            last_finish = job['finish']

    return selected
```

Listing 7: Greedy earliest-finish-time scheduling (src/d1_greedy_scheduling.py).

Output:

```
All jobs sorted by finish time:
Job      Start    Finish   Reward
------------------------------------
```

```
J1        1         4         5
J2        3         5         2
J3        0         6         8
J4        4         7         6
J5        3         8         7

Selected jobs (2 non-overlapping):
Job       Start     Finish    Reward
------------------------------------
J1        1         4         5
J4        4         7         6

Jobs selected : 2
Total reward  : 11
```

The greedy algorithm selects J1 (finishes earliest at time 4) then J4 (starts exactly at 4, compatible), yielding the maximum possible count of 2 non-overlapping jobs. J2, J3, and J5 all overlap with J1.

D2 — Weighted Interval Scheduling (DP)

Recurrence. Sort jobs by finish time and index them $1 \ldots n$. Let $p(j)$ be the index of the latest job $i < j$ with $\text{finish}(i) \leq \text{start}(j)$ (0 if none). Define:

$$\text{OPT}(0) = 0, \qquad \text{OPT}(j) = \max\big(\text{reward}_j + \text{OPT}(p(j)),\ \text{OPT}(j-1)\big).$$

The first argument includes job j; the second excludes it.

```python
def compute_predecessors(jobs):
    n = len(jobs)
    p = [-1] * n
    for j in range(n):
        for i in range(j - 1, -1, -1):
            if jobs[i]['finish'] <= jobs[j]['start']:
                p[j] = i
                break
    return p


def weighted_interval_scheduling(jobs):
    jobs_sorted = sorted(jobs, key=lambda j: j['finish'])
    n  = len(jobs_sorted)
    p  = compute_predecessors(jobs_sorted)

    dp = [0] * (n + 1)
    for j in range(1, n + 1):
        job         = jobs_sorted[j - 1]
        pred_dp_idx = p[j - 1] + 1          # shift 0-indexed p to 1-indexed dp
        dp[j] = max(job['reward'] + dp[pred_dp_idx], dp[j - 1])

    # Reconstruction: walk backwards
    selected = []
    j = n
    while j >= 1:
        job         = jobs_sorted[j - 1]
        pred_dp_idx = p[j - 1] + 1
        if job['reward'] + dp[pred_dp_idx] >= dp[j - 1]:
            selected.append(job)
            j = pred_dp_idx
        else:
            j -= 1
    selected.reverse()
```

```
    return dp[n], selected, dp, jobs_sorted, p
```

Listing 8: Predecessor computation and DP (src/d2_weighted_dp.py).

Output:

```
Jobs sorted by finish time (p uses 1-based indexing, 0 = none):
j      Job       Start     Finish    Reward    p(j)
-------------------------------------------------
1      J1        1         4         5         0
2      J2        3         5         2         0
3      J3        0         6         8         0
4      J4        4         7         6         1
5      J5        3         8         7         0

Recurrence:
OPT(0) = 0
OPT(j) = max( reward_j + OPT(p(j)),  OPT(j-1) )

DP table:
j       OPT(j)
--------------
0       0
1       5
2       5
3       8
4       11
5       11

Maximum total reward: 11

Selected jobs:
Job       Start     Finish    Reward
-------------------------------------
J1        1         4         5
J4        4         7         6

Total reward: 11
```

DP table walkthrough.

- $\mathrm{OPT}(1) = \max(5+0,\, 0) = 5$ (include J1).
- $\mathrm{OPT}(2) = \max(2+0,\, 5) = 5$ (exclude J2; its reward alone is too small).
- $\mathrm{OPT}(3) = \max(8+0,\, 5) = 8$ (include J3 alone gives $8 > 5$).
- $\mathrm{OPT}(4) = \max(6+\mathrm{OPT}(1),\, \mathrm{OPT}(3)) = \max(11, 8) = 11$ (J4 is compatible with J1).
- $\mathrm{OPT}(5) = \max(7+0,\, 11) = 11$ (exclude J5).

Reconstruction. Starting from $j = 5$: $7 < 11$, exclude J5, $j = 4$. At $j = 4$: $11 \geq 8$, include J4, jump to $j = p(4) = 1$. At $j = 1$: $5 \geq 0$, include J1, jump to $j = 0$. Stop. Selected: J1, J4, total reward 11.

D3 — Greedy Failure Example for Weighted Scheduling

Counterexample. Consider two jobs:

Job	Start	Finish	Reward
A	1	2	1
B	1	10	100

Both jobs start at time 1 and are mutually incompatible (overlapping).

- **Greedy (earliest finish, D1 strategy).** Sorts: A finishes at 2, B at 10. Selects A (finishes earlier); then B starts at $1 < 2$, so skip. **Result:** $\{A\}$, count = 1, reward = 1.
- **DP (weighted, D2 strategy).** $\mathrm{OPT}(1) = \max(1, 0) = 1$ (take A). $\mathrm{OPT}(2) = \max(100 + 0, 1) = 100$ (take B). **Result:** $\{B\}$, reward = 100.

Explanation of the difference. The two objectives are fundamentally different:

- **Maximise job count.** Greedy by earliest finish is optimal: choosing the job that finishes soonest leaves the maximum remaining time for future jobs.
- **Maximise total reward.** Greedy by earliest finish can be arbitrarily bad: it may pick a low-reward job that finishes early, blocking a high-reward job. In the example above, greedy earns reward 1 while the optimal earns 100 – a factor-of-100 loss.

Dynamic programming is required for the weighted version because the optimal solution may skip short, cheap jobs in favour of longer, more rewarding ones – a trade-off that greedy cannot capture.